

SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 2

ERROR ANALYSIS TASK

Name of the Student	G VENKATA GANESH
Reg. No	192425381

COURSE NAME: Deep Learning
FACULTY : Dr.Arul Raj A.M

COURSE CODE: MLA0403

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15%

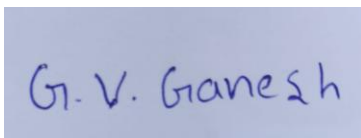
Assessment Tool Weightage: 30%

Duration: 30 minutes

Marks: 25

Code of Conduct:

I G VENKATA GANESH / 192425381 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

QUESTIONS:**Total Marks:25****1.Error Analysis in Maximum Likelihood Estimation (MLE)**

A student builds a binary classifier using Maximum Likelihood Estimation. The likelihood function is incorrectly written as the sum of probabilities instead of the product. The model produces poor parameter estimates even with large data.

Tasks:

a) Identify the conceptual error in the likelihood formulation.

Answer :

The conceptual error is writing the likelihood as the sum of probabilities instead of the product of probabilities. In MLE, observations are assumed to be independent, so the joint probability is obtained by multiplying the individual probabilities, not adding them.

b) Explain how this mistake affects parameter estimation.

Answer :

Using the sum of probabilities does not correctly represent the probability of observing the entire dataset. As a result:

- The optimization objective becomes mathematically incorrect.
- Estimated parameters do not maximize the true likelihood.
- The model produces biased and inaccurate predictions, even with large datasets.
- Increasing the amount of data cannot compensate for the incorrect formulation.

c) Suggest the correct mathematical formulation and reasoning.

Answer:

For independent observations $x_1, x_2, \dots, x_n$:

Likelihood:

$$L(\theta) = \prod_{i=1}^n P(x_i | \theta)$$

The correct objective is to find:

$$\hat{\theta} = \arg \max_{\theta} L(\theta)$$

This formulation correctly represents the probability of observing all data simultaneously.

d) How would using log-likelihood fix computational issues?

Answer :

The log-likelihood is:

$$\log L(\theta) = \sum_{i=1}^n \log P(x_i | \theta)$$

Advantages:

- Converts products into sums, simplifying optimization.
- Prevents numerical underflow caused by multiplying very small probabilities.
- Makes gradient computation easier.
- Improves computational efficiency and stability.

2. Error Diagnosis in Perceptron Learning

While implementing a Perceptron, a student observes that the model fails to converge even after many iterations on a linearly separable dataset.

Tasks:

a) List possible implementation errors in the learning algorithm.

Answer :

Possible implementation errors include:

- Incorrect weight update formula.
- Missing or incorrect bias update.
- Wrong prediction threshold.
- Incorrect sign in the update rule.
- Errors in feature normalization.
- Improper stopping criteria.
- Random initialization errors.

b) Analyze how incorrect learning rate or weight updates affect convergence.

Answer :

- A very high learning rate causes overshooting and oscillation.
- A very small learning rate results in extremely slow convergence.
- Incorrect weight update direction increases classification errors instead of reducing them.
- Failure to update bias prevents finding the correct decision boundary.

c) Identify dataset-related issues that may cause this problem.

Answer :

Possible dataset issues:

- Incorrect class labels.
- Duplicate or noisy samples.
- Features with vastly different scales.
- Hidden non-linearity despite assumptions.
- Data preprocessing mistakes.

d) Propose modifications to ensure convergence.

Answer :

- Normalize or standardize input features.
- Choose a suitable learning rate.
- Correctly implement weight and bias updates.
- Shuffle training samples each epoch.
- Verify labels and preprocessing.
- Use the Perceptron update rule correctly:

$$w = w + \eta(y - \hat{y})x$$

where η is the learning rate.

3. Backpropagation Gradient Error Analysis

A Multilayer Perceptron trained using Backpropagation shows increasing loss during training instead of decreasing.

Tasks:

a) Identify possible mathematical or coding errors in gradient computation.

Answer :

Possible errors include:

- Incorrect derivative calculations.
- Wrong chain rule implementation.
- Incorrect sign during gradient updates.
- Updating weights before computing all gradients.
- Indexing or matrix multiplication mistakes.
- Incorrect loss function implementation.

b) Explain the role of activation functions in gradient flow.

Answer :

Activation functions introduce non-linearity into neural networks.

- Sigmoid and tanh may saturate, causing very small gradients.
- ReLU maintains stronger gradients for positive inputs.
- Appropriate activation functions help gradients propagate efficiently through deep networks.

c) Analyze how vanishing or exploding gradients affect training.

Answer :

Vanishing Gradients

- Gradients become extremely small.
- Early layers learn very slowly.
- Training stagnates.

Exploding Gradients

- Gradients become excessively large.
- Weight updates become unstable.
- Loss increases rapidly and training diverges.

d) Suggest techniques to fix the issue (e.g., normalization, initialization).

Answer :

Recommended techniques:

- Xavier or He weight initialization.
- Batch Normalization.
- Gradient Clipping.
- Use ReLU or Leaky ReLU activation.
- Reduce learning rate.
- Apply proper input normalization.
- Use Adam or RMSProp optimizer.

4. Gradient Descent vs SGD Error Investigation

A model trained using Gradient Descent converges very slowly, while the same model with Stochastic Gradient Descent shows unstable loss fluctuations.

Tasks:

a) Explain why batch gradient descent is slow in large datasets.

Answer :

Batch Gradient Descent computes gradients using the entire dataset before every update. Therefore:

- Each iteration is computationally expensive.
- Requires significant memory.
- Training becomes slow for large datasets.

b) Analyze the cause of fluctuations in SGD.

Answer :

Stochastic Gradient Descent updates weights after every training sample.

This causes:

- Noisy gradient estimates.
- Frequent oscillations in loss.
- Faster but less stable convergence.

c) Compare the error behavior of Mini-Batch Gradient Descent.

Answer :

Method	Error Behaviour
Batch GD	Smooth, stable convergence but slow
SGD	Fast convergence with noisy fluctuations
Mini-Batch GD	Balanced approach with moderate fluctuations and faster convergence

d) Recommend the best approach and justify with error trade-offs.

Answer :

Mini-Batch Gradient Descent is generally the best choice because:

- Faster than Batch Gradient Descent.
- More stable than SGD.
- Efficient GPU utilization.
- Lower computational cost.
- Better convergence speed with acceptable noise.

5. Curse of Dimensionality Error Analysis

A machine learning model built using high-dimensional data performs well on training data but poorly on test data due to overfitting, related to the Curse of Dimensionality.

Tasks:

a) Explain how high dimensionality increases model error.

Answer :

As the number of features increases:

- Data becomes sparse.
- More training samples are required.
- The model learns noise instead of meaningful patterns.
- Generalization performance decreases.
- Test error increases due to overfitting.

b) Identify signs of overfitting in this scenario.

Answer :

Common signs include:

- Very high training accuracy.
- Low testing accuracy.
- Large gap between training and testing performance.
- Poor prediction on unseen data.
- High variance in model performance.

c) Analyze why distance-based learning fails in high dimensions.

Answer :

In high-dimensional spaces:

- Distances between points become increasingly similar.
- Nearest neighbors become difficult to distinguish.
- Similarity measures lose effectiveness.
- Algorithms such as KNN and clustering produce poor results.

d) Suggest dimensionality reduction or regularization techniques to improve performance.

Answer :

Dimensionality Reduction

- Principal Component Analysis (PCA)
- Linear Discriminant Analysis (LDA)
- Autoencoders
- Feature Selection

Regularization

- L1 (Lasso)
- L2 (Ridge)
- Elastic Net
- Dropout (for neural networks)

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand